

Coupon Collector

Assuming that $n = 100$, we simulate the *Coupon Collector* problem, by implementing the corresponding algorithm in MATLAB.

We can now run the algorithm for an adequate number of times ($K = 500$) and on each iteration obtain the number of "boxes of cereal" (as demonstrated in the notes) that had to be opened, in order to acquire every single one of the n coupons available.

Focusing on the results of the plot figure, one can easily see that the number of iterations is "oscillating" up and down of a line, representing the average iterations needed (highlighted in orange), on $y \approx 515$ (#iterations).

Also, in regards to the histogram, we can say that if we were to find a spot on the histogram that the total mass of the bars would be equally divided, this spot would be somewhere around the 500 mark. That's because although bars of higher mass (frequency) are on the left of 500, there are more low-mass bars on its right, thus balancing out the total mass of the histogram, around this point.

In other words, a somewhat precise estimation on where the mean value of required selections is located, would be the 515 #iterations.

The expected value of boxes having to be opened in order to get all n coupons is given by $n \ln n + \Theta(n)$. This means that, the expected value of iterations is $n \ln n$ plus a tightly bounded linear term.

The results returned by the MATLAB simulation of the Coupon Collector problem confirm the formula given previously, since we have estimated the expected value of iterations to be ~ 515 , which we can say that (for $n = 100$) it is $100 \cdot \ln 100 \approx 460$ plus a term tightly bounded from a linear term.

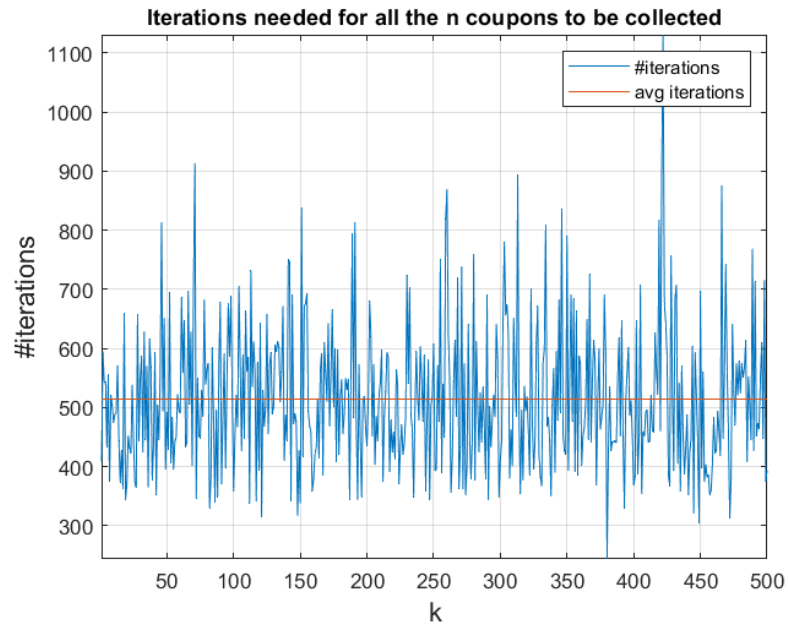


Figure 1: Plot of the required selections versus k

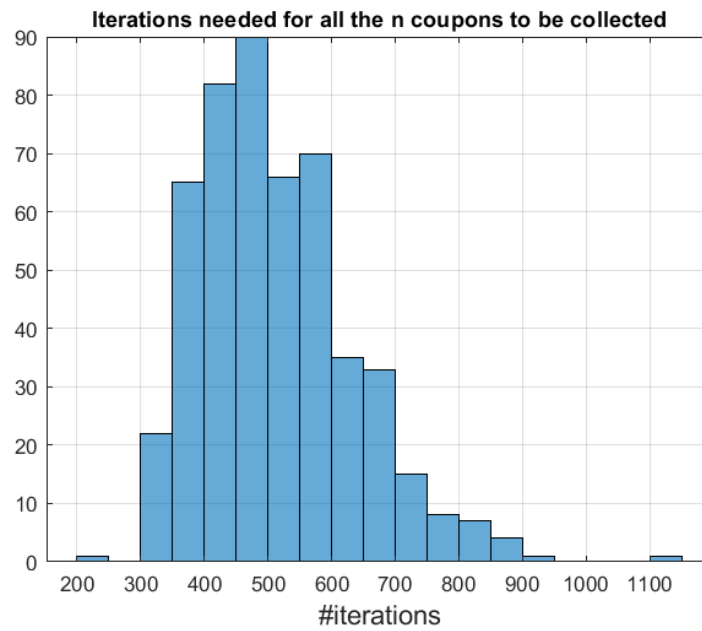


Figure 2: Histogram of the number of the required selections

Quick Sort

We now implement the *Quick Sort* algorithm and choose a number of times for it to be ran ($K = 500$), in order to get a good sense of how it behaves in terms of comparisons made. The number of elements n to be sorted is chosen to be equal to a power of 2, just for convenience in the thinking behind the implementation of the algorithm, and it will not affect the results derived from the experiment.

As it is clearly visible, the number of comparisons for all the iterations tend to be averaged out to the orange line highlighted in the plot, on roughly 12645 comparisons. Moreover, when it comes to the histogram plotted, using the same intuition from the previous exercise, one may confirm that the average number of comparisons made in the execution of the algorithm is around 12600.

Since there is no standard way of choosing the pivot each element has the same probability of being assigned as the pivot value. This means that, throughout the K iterations that the algorithm is ran, there will be cases that the pivot is chosen "poorly", such that it separates the list in two sub-lists of 1 and $n - 1$ elements and cases that it will be chosen optimally in a way that the two sub-lists are equal in size. This means that, there is no meaning in comparing the worst nor the optimal case of the complexity that occurs, but instead of the average case complexity.

From the notes, it is known that $T(n) \leq 2n \ln n$. Indeed, when substituting $n = 2^{10}$ in the formula above, we get $T(n) \leq 2 \cdot 2^{10} \ln 2^{10} \approx 14195$ which is pretty close to what the average comparisons were estimated to be, from the experiment (12645 #comparisons).

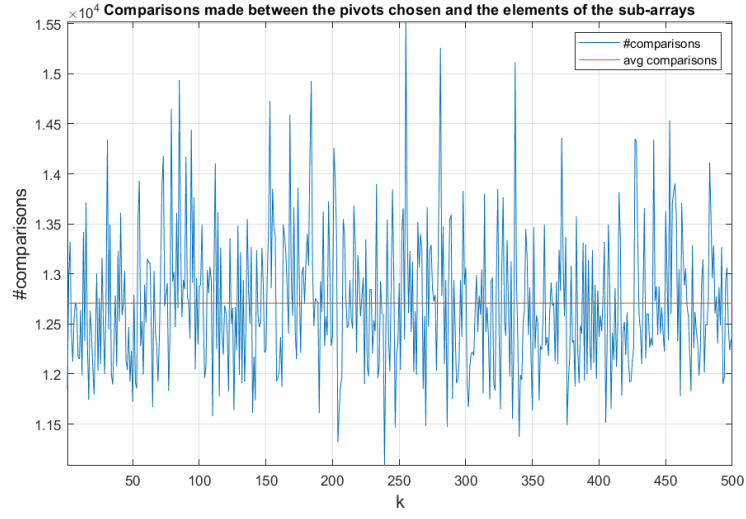


Figure 3: Plot of the comparisons made at the k -th execution of the Quick Sort

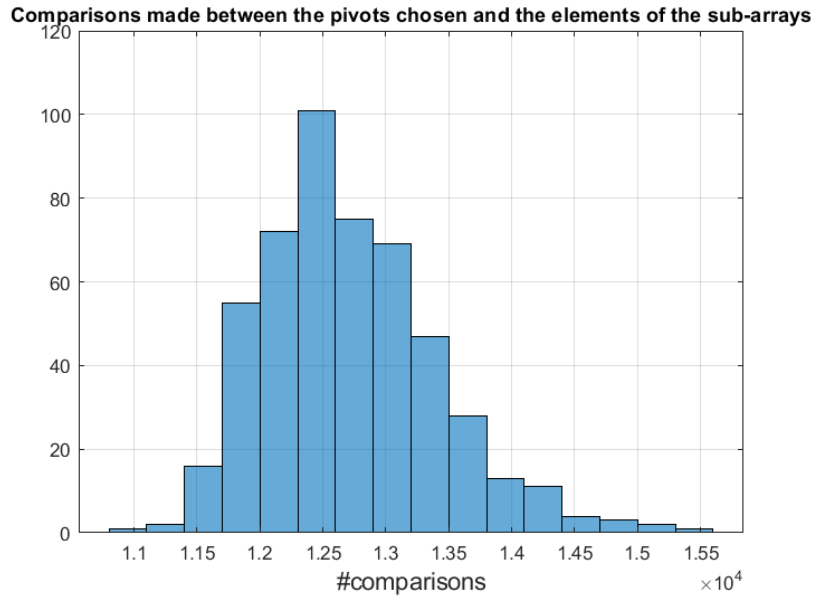


Figure 4: The distribution of comparisons made in K iterations of Quick Sort, in Histogram

Randomized Median

We implement the randomized algorithm and keep the number of comparisons, as well as the the median that corresponds to each successful iteration of the algorithm for every value of n .

- (a). Below, there is the table containing every mean calculated for every of the 50 fixed values of n , for a certain execution of the experiment. It is obvious, that because of the randomly selected elements on every iteration of the algorithm, when the very same program gets executed, it is likely that the randomized mean for, say $n = 200$ elements will be different each time the program was ran.

n	200	400	600	800	1000	1200	1400	1600	1800	2000
Mean	66	109	106	162	135	158	201	180	218	223

n	2200	2400	2600	2800	3000	3200	3400	3600	3800	4000
Mean	321	351	257	236	345	330	274	347	426	373

n	4200	4400	4600	4800	5000	5200	5400	5600	5800	6000
Mean	506	383	614	450	733	574	538	416	596	438

n	6200	6400	6600	6800	7000	7200	7400	7600	7800	8000
Mean	481	647	685	620	826	741	521	616	593	716

n	8200	8400	8600	8800	9000	9200	9400	9600	9800	10000
Mean	544	569	647	858	904	712	902	689	616	790

Table 1: Table with the calculated means for a certain run of the program

- (b). In the following plot, we depict the #comparisons made in every iteration vs the number of the iteration n .



Figure 5: #comparisons vs n

In essence, the algorithm never fails, since in case of failure, we keep executing it for a given n , until it returns an arithmetic value for the median. Also, the result of a certain iteration does not contribute in any way to the result of another iteration, meaning that results of different iterations are independent of one another. It is also known that the number of runs until success is given by a geometric r.v. giving a running time of linear complexity, when it comes to the number of runs. Given all the above, since on every iteration (fixed n) a number of comparisons c is made, the total running time given in terms of the total comparisons will just be a multiple c of the number of successes. Therefore, since constant terms can be ignored in asymptotic studying, the running time in terms of total comparisons made will be of linear form too.

As we can clearly see, the algorithm is of linear complexity, indeed.